MINI PROJECT REPORT

ON

Design and Development of a Smart Event Management System Using Fuzzy Logic

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING

BY

Madhavi Raolani (T2, 2401091)

Janhvi Ramrakhyani (T2, 2401089)

SUBJECT

AGILE SOFTWARE DEVELOPMENT AND DEVOPS

SUPERVISOR

Prof. Sanober Shaikh



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE
THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050
UNIVERSITY OF MUMBAI
[2025-26]



THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2025-26

Certificate

This is to certify that the Mini Project entitled **Design and Development of a Smart Event Management System Using Fuzzy Logic** is a bonafide work of **Madhavi Raolani** and **Janhvi Ramrakhyani** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **Undergraduate** in **Artificial Intelligence and Data Science**.

_____

Prof. Sanober Shaikh
(Supervisor)

Abstract

Choosing which event to attend is rarely a matter of exact values. Attendees think in vague terms such as *mellow*, *after work*, *nearby* or *into tech*, while most event listings answer only with hard filters on category, date and distance. A crisp rule such as "show events within 5 km" treats an event at 4.9 km and one at 5.1 km completely differently, and returns nothing at all when no event fits every condition exactly.

This mini project, **"Design and Development of a Smart Event Management System Using Fuzzy Logic"**, addresses this gap by building **AuraEvents**, a smart event matcher. The attendee sets a vibe energy level, a preferred time of day, a maximum travel distance and an interest tag on a web dashboard. A Python (FastAPI) backend converts these inputs into degrees of membership in fuzzy sets, combines them with weights of 45% vibe, 35% timing and 20% distance, lets the interest tag modulate the result, and returns every event with a percentage match and a label such as *Perfect Vibe* or *Balanced Fit*, ranked from best to worst.

The system is built and delivered using **Agile Scrum** and **DevOps** practices. Work was organized into three time-boxed sprints: infrastructure and UI scaffolding, the fuzzy logic and backend core, and a CI/CD pipeline with monitoring. The application, a Jenkins server and a Prometheus server run as containers under **Docker Compose**. A declarative **Jenkins** pipeline triggered by a **GitHub** webhook rebuilds and redeploys the containers on every push to the main branch, and **Prometheus** scrapes the application's metrics endpoint every five seconds, including a custom counter of match requests and a histogram of fuzzy inference time. Jira is used for the backlog and sprints.

In five test scenarios the fuzzy engine always produced a complete ranking, whereas a crisp exact-match filter returned no event at all in two of them and only one event in the other three. The result shows how soft computing can make recommendations more natural, and how Agile and DevOps practices allow such a system to be built, deployed and observed in short iterations.

Keywords: *Fuzzy Logic, Event Recommendation, Membership Functions, FastAPI, Agile Scrum, Docker, Jenkins CI/CD, Prometheus*

TABLE OF CONTENTS

Content **Page No.**

Abstract 3
List of Figures 5
1. Introduction 6
1.1 Background 6
1.2 Problem Statement 6
2. Implementation 8
2.1 Technologies Used 8
2.2 Code Structure 8
2.3 Fuzzy Inference Design 9
2.4 Sprint-wise Implementation (Agile Scrum) 10
2.5 CI/CD and Monitoring Pipeline 10
3. Results and Discussion 13
3.1 Result 13
3.2 Discussion 15
4. Conclusion 17
4.1 Conclusion 17
4.2 Future Scope 17
5. References 18

List of Figures

Fig. No.	Title	Page No.
2.1	Architecture of AuraEvents: CI/CD flow and runtime services	11
3.1	AuraEvents dashboard: preference indicators and Smart Match Feed	13
3.2	Dashboard after changing preferences (cards re-ranked by the fuzzy engine)	14
3.3	JSON response of the POST /matches endpoint	15
3.4	Prometheus target status and the total_matches_requested_total query	15
3.5	Jenkins pipeline build triggered by a GitHub push	15

Chapter 1
Introduction

1.1 Background

Colleges, communities and cities host a continuous stream of events: workshops, meetups, cultural evenings, wellness sessions and tech talks. With many events running at the same time, the question for an attendee is not whether something is happening but which event suits *this* person at *this* time. Most event platforms answer with category tags, date pickers and a distance filter. These filters are crisp: an event either passes or it does not.

Human preferences, however, are inherently imprecise. A person who wants a *calm evening nearby* has no exact energy value, exact hour or exact radius in mind. Classical (crisp) logic cannot represent such statements well. A rule like "show events within 5 km" treats an event at 4.9 km and one at 5.1 km completely differently, and when no event satisfies every condition the user is shown an empty page instead of the next-best options.

Fuzzy logic, introduced by Lotfi A. Zadeh in 1965 [1], handles this kind of vagueness. Instead of true or false, an element belongs to a set with a degree of membership between 0 and 1. Combined with linguistic IF–THEN rules and an inference mechanism such as the one proposed by Mamdani and Assilian [2] or the weighted-average model of Takagi and Sugeno [7], it lets a system reason with expressions like "the event's vibe is *similar* to what I want" or "the venue is *near enough*" and still produce a precise, usable output [5]. Fuzzy systems are easy to interpret and need no training dataset, which makes them attractive for decision-support features.

On the process side, event platforms evolve constantly: new filters, new event types, new data sources. Agile methods, formalized in the Agile Manifesto [4], suit such an environment. Scrum [3], the most widely used Agile framework, builds on the team-based approach described by Takeuchi and Nonaka [6] and organizes development into short time-boxed sprints, each delivering a working increment. DevOps extends the same idea to delivery and operations: code is built, deployed and monitored automatically so that every increment actually reaches users and problems become visible through metrics.

This project brings these ideas together. Fuzzy logic is used inside the system to match attendees with events, and Agile Scrum with a Docker, Jenkins and Prometheus toolchain is used as the process and platform to build, ship and observe it.

1.2 Problem Statement

Attendees choosing among many events rely on rigid category, time and distance filters. These filters cannot express graded preferences such as "mellow", "evening" or "nearby", cannot rank near-misses, and often return no result at all. In addition, such a service is only useful if it can be changed quickly, deployed reliably and monitored once it is running.

The problem addressed in this project is to design and develop a smart event management system that:

- accepts imprecise attendee preferences (vibe energy, preferred timing, maximum distance and interest) through an interactive dashboard;
- uses fuzzy logic to convert those preferences into a percentage match for every event and returns a ranked list, even when no event is a perfect fit;
- is built iteratively using Agile Scrum, packaged with Docker Compose, deployed automatically by a Jenkins pipeline triggered from GitHub, and monitored through Prometheus metrics.

Objectives

1. To study fuzzy logic concepts: fuzzy sets, membership functions, aggregation of rules and weighted-average defuzzification.
2. To design a fuzzy inference engine that computes a percentage match between an attendee's preferences and each event.
3. To develop a FastAPI backend exposing `/matches`, `/health` and `/metrics` endpoints, and a web dashboard that calls it.
4. To apply Agile Scrum (product backlog in Jira, three time-boxed sprints, a working increment per sprint) with Git and GitHub for version control.
5. To automate build and deployment with Docker Compose and a Jenkins declarative pipeline triggered by a GitHub webhook, and to monitor the application with Prometheus.
6. To test the system on sample scenarios and compare fuzzy matching with a simple crisp-threshold filter.

Chapter 2
Implementation

2.1 Technologies Used

Table 2.1 lists the technologies used in each layer of the system.

Table 2.1: Technology stack

Layer	Technology	Purpose
Language	Python 3.12	Backend logic and the fuzzy engine
Backend framework	FastAPI, Uvicorn, Pydantic	REST API (`/matches`, `/health`, `/metrics`) with request validation; also serves the dashboard
Fuzzy engine	Pure Python (math module)	Gaussian and piecewise-linear membership functions, weighted aggregation
Front end / UI	HTML, Tailwind CSS, JavaScript (fetch API); Snitch design references	Responsive pastel dashboard with sliders and interest badges
Containerization	Docker, Docker Compose	Three services (app, Jenkins, Prometheus) on one network
CI / CD	Jenkins (declarative Jenkinsfile), GitHub webhook	Automatic build and deployment on every push to main
Monitoring	Prometheus, prometheus_client	5-second scraping; custom counter and histogram
Scrum tools	Jira	Product backlog and sprint board
Version control	Git and GitHub	Source control; source of the webhook
Testing	curl, Prometheus HTTP API	Endpoint and metric verification

2.2 Code Structure

```
aura-events/
├── docker-compose.yml        # app, jenkins, prometheus services
├── Jenkinsfile               # declarative CI/CD pipeline
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app/
│       ├── __init__.py
│       └── main.py           # fuzzy engine, /matches, /health, /metrics
├── frontend/
│   └── index.html            # dashboard UI + fetch() integration
├── jenkins/
│   └── Dockerfile            # Jenkins image with Docker CLI + Compose
├── prometheus/
│   ├── Dockerfile
│   └── prometheus.yml        # scrape configuration (every 5 s)
├── .dockerignore
└── .gitignore
```

2.3 Fuzzy Inference Design

The fuzzy engine lives in `backend/app/main.py` and scores every event against the attendee's current preferences. The inputs come from the dashboard: two sliders on a 1–3 scale (vibe energy and preferred timing), a distance slider (2–25 km) and an interest badge. Each input is fuzzified into a membership degree between 0 and 1, as summarized in Table 2.2.

Table 2.2: Inputs, membership functions and roles

Input	Range	Fuzzy set and membership function	Role
Vibe energy	1 (mellow) to 3 (high energy)	"Similar": Gaussian, $\mu = \exp(-d^2 / 2\sigma^2)$, $\sigma = 1$, $d = \lvert user - event \rvert$	Weight 0.45
Preferred timing	1 (morning) to 3 (evening)	"Similar": same Gaussian function	Weight 0.35
Max distance	2 to 25 km	"Near enough" with r = distance / limit: $\mu = 1 - 0.3r$ for $r \leq 1$; $\mu = 0.7(1.5 - r)/0.5$ for $1 < r \leq 1.5$; 0 beyond	Weight 0.20
Interest	all, tech, casual, creative, wellness, food (optional tech level 0–100)	Tag match: 1.0 if the tag equals the event category or "all", otherwise 0.35. If a tech level is given, it is averaged with a Gaussian ($\sigma = 0.4$)	Modulator

The matching rule can be stated in words as: **IF** the vibe is similar **AND** the timing is similar **AND** the distance is near enough **THEN** the match is high. The three membership degrees are aggregated by a weighted average, and the interest degree then scales the result:

$$base = 0.45 \times \mu_vibe + 0.35 \times \mu_time + 0.20 \times \mu_distance$$

$$match = base \times (0.7 + 0.3 \times \mu_interest)$$

$$percentage = \min(99, \operatorname{round}(100 \times match))$$

The percentage is turned into a label: 90% and above is *Perfect Vibe*, 75–89% is *Balanced Fit* and anything lower is *Gentle Fit*. Because the interest factor lies between 0.7 and 1, an event outside the chosen category is scaled down to about 80% of its score rather than removed, which keeps near-misses visible. The weights of 45/35/20 are the same ones shown on the dashboard's "Fuzzy Weights Synced" bar.

Instead of a full Mamdani rule base with centroid defuzzification, the engine uses a weighted average of membership degrees, which is similar in spirit to a zero-order Takagi–Sugeno system [7] and keeps inference time in the millisecond range. The weights and constants are design parameters chosen by the team; they were not learned from user data.

The backend ranks a small mock database of three events (Table 2.3), one for each card on the dashboard.

Table 2.3: Mock event database

Event	Category	Vibe	Time slot	Distance
Pastel Pottery & Herbal Matcha Evening	creative	1 (mellow)	3 (evening)	2.4 km
Ambient AI & UI Builders Fireside	tech	2 (balanced)	3 (evening)	4.1 km
Botanical Garden Breathwork & Sound Bath	wellness	1 (mellow)	1 (morning)	5.8 km

Worked example. For the default preferences (vibe 1, evening, 8 km, all interests), the pottery evening has $\mu_vibe = 1.00$, $\mu_time = 1.00$ and $\mu_distance = 1 - 0.3 \times (2.4/8) = 0.91$, so base $= 0.45 + 0.35 + 0.20 \times 0.91 = 0.982$ and the match is 98%. The AI fireside has $\mu_vibe = \exp(-0.5) = 0.607$, $\mu_time = 1.00$ and $\mu_distance = 0.846$, so base $= 0.273 + 0.35 + 0.169 = 0.792$ and the match is 79%.

2.4 Sprint-wise Implementation (Agile Scrum)

The product was built in three time-boxed sprints inside a two-hour build window. The project manager acted as product owner and set each sprint goal, the team delivered a working increment per sprint, and Jira issue keys (AURA-1, AURA-2 and AURA-3) were used in commit messages so that GitHub commits link back to the Jira backlog. Representative backlog items were: *as an attendee, I want to set my preferred vibe so that I see events that suit my mood*; *as the team, we want every push to main to deploy automatically so that the running system always matches the code*; and *as the team, we want request counts and inference time exposed as metrics so that we can see how the system is used*.

Table 2.4: Sprint plan and deliverables

Sprint	Goal	Main deliverables
Sprint 1	Infrastructure and UI scaffolding	docker-compose.yml with app, Jenkins and Prometheus services; FastAPI skeleton serving the dashboard with `/health` and `/metrics`; pastel dashboard UI; Git repository linked to Jira
Sprint 2	Fuzzy logic and backend core	Fuzzy scoring engine; `POST /matches` with input validation and CORS; custom Prometheus metrics; JavaScript `fetch()` wiring of sliders and badges to the API
Sprint 3	CI/CD pipeline and monitoring	Jenkinsfile with GitHub-push trigger; final Prometheus configuration (5-second scrape); Docker images revised for automated deployment; verification commands

2.5 CI/CD and Monitoring Pipeline

Figure 2.1 shows how the pieces fit together. A push to the main branch on GitHub fires a webhook to Jenkins, which runs the pipeline defined in the repository's `Jenkinsfile`. The pipeline uses the Docker socket of the host to build and start the containers, and Prometheus then scrapes the freshly deployed application.

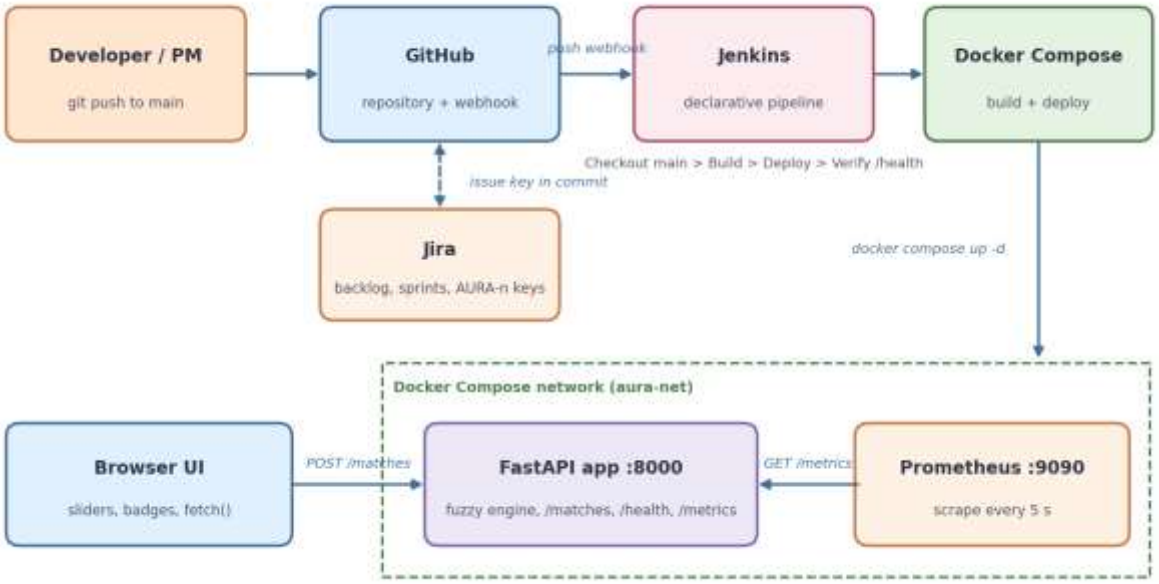


Fig. 2.1 Architecture of AuraEvents: CI/CD flow and runtime services

The declarative pipeline has four stages:

- **Checkout main:** checks out the `main` branch of the GitHub repository.
- **Build containers:** runs `docker compose build app prometheus` to build both images.
- **Deploy:** runs `docker compose up -d app prometheus` to start or recreate the services.
- **Verify deployment:** polls `http://app:8000/health` until the application answers, and fails the build otherwise.

The trigger is `githubPush()`, with a `pollSCM` schedule every two minutes as a fallback if the webhook cannot reach Jenkins. Three design decisions were needed to make containerized deployment from inside Jenkins work reliably:

- The stock Jenkins image has no Docker client, so a custom Jenkins image adds the Docker CLI and the Compose plugin.
- Bind mounts resolve on the host rather than inside Jenkins' workspace, so the frontend and the Prometheus configuration are copied into their images instead of being mounted.
- The pipeline deploys only the `app` and `prometheus` services; recreating the Jenkins container from inside its own pipeline would terminate the build. A fixed Compose project name keeps manual runs and Jenkins runs on the same containers.

Prometheus scrapes the application every five seconds. Table 2.5 lists the metrics exposed at `/metrics`.

Table 2.5: Metrics exposed by the application

Metric	Type	Purpose
`total_matches_requested_total{interest}` label	Counter	Number of `/matches` requests, labelled by the selected interest badge (exposed with the `_total` suffix added by the client library)
`fuzzy_inference_seconds`	Histogram	Time spent in the fuzzy scoring step
`http_requests_total`	Counter	All HTTP requests by method, route and status
`http_request_duration_seconds`	Histogram	Request latency by method and route
`aura_app_up`	Gauge	Set to 1 while the application is running

Chapter 3
Results and Discussion

3.1 Result

The final system runs as three containers started with `docker compose up`. The dashboard is served by the FastAPI application on port 8000, Jenkins is available on port 8080 and Prometheus on port 9090. Figure 3.1 shows the dashboard: the left column holds the preference indicators (vibe energy, preferred timing, maximum distance and interest badges) and the right column holds the Smart Match Feed with one card per event, each showing its fuzzy match percentage.

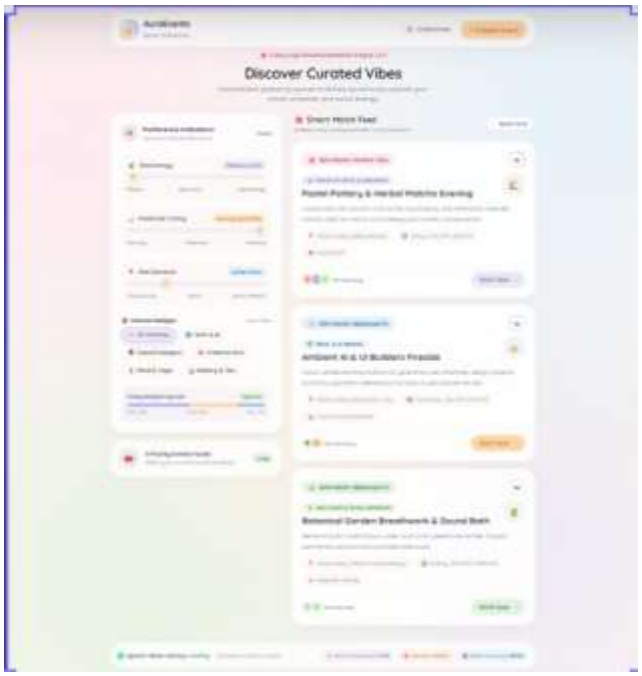


Fig. 3.1 AuraEvents dashboard: preference indicators and Smart Match Feed

To evaluate the engine, five preference scenarios were scored with the implemented algorithm and compared with a crisp baseline. The crisp baseline returns an event only if its vibe and time slot equal the requested ones exactly, its distance is within the limit and its category matches the chosen interest. Table 3.1 gives the results.

Table 3.1: Fuzzy ranking versus crisp filter

Scenario	Preferences (vibe, timing, max km, interest)	Fuzzy ranking (match %)	Crisp filter
S1	Mellow, evening, 8 km, all	Pottery 98 · AI Fireside 79 · Botanical 65	Pottery only
S2	Balanced, evening, 15 km, tech (tech level 90)	AI Fireside 98 · Pottery 63 · Botanical 38	AI Fireside only
S3	Mellow, morning, 8 km, wellness	Botanical 96 · Pottery 55 · AI Fireside 39	Botanical only
S4	High energy, evening, 3 km, all	AI Fireside 66 · Pottery 56 · Botanical 11	No event
S5	Balanced, afternoon, 5 km, casual	AI Fireside 65 · Pottery 53 · Botanical 47	No event

Table 3.2 shows how the scores react when only the distance limit changes, using the S1 preferences.

Table 3.2: Effect of the distance limit on match percentage (S1 preferences)

Max distance	Pottery (2.4 km)	AI Fireside (4.1 km)	Botanical (5.8 km)
2 km	88	62	50
3 km	95	66	50
5 km	97	77	59
8 km	98	79	65

Figures 3.2 to 3.5 show the dashboard after a preference change, the API response, the Prometheus view and the Jenkins build.

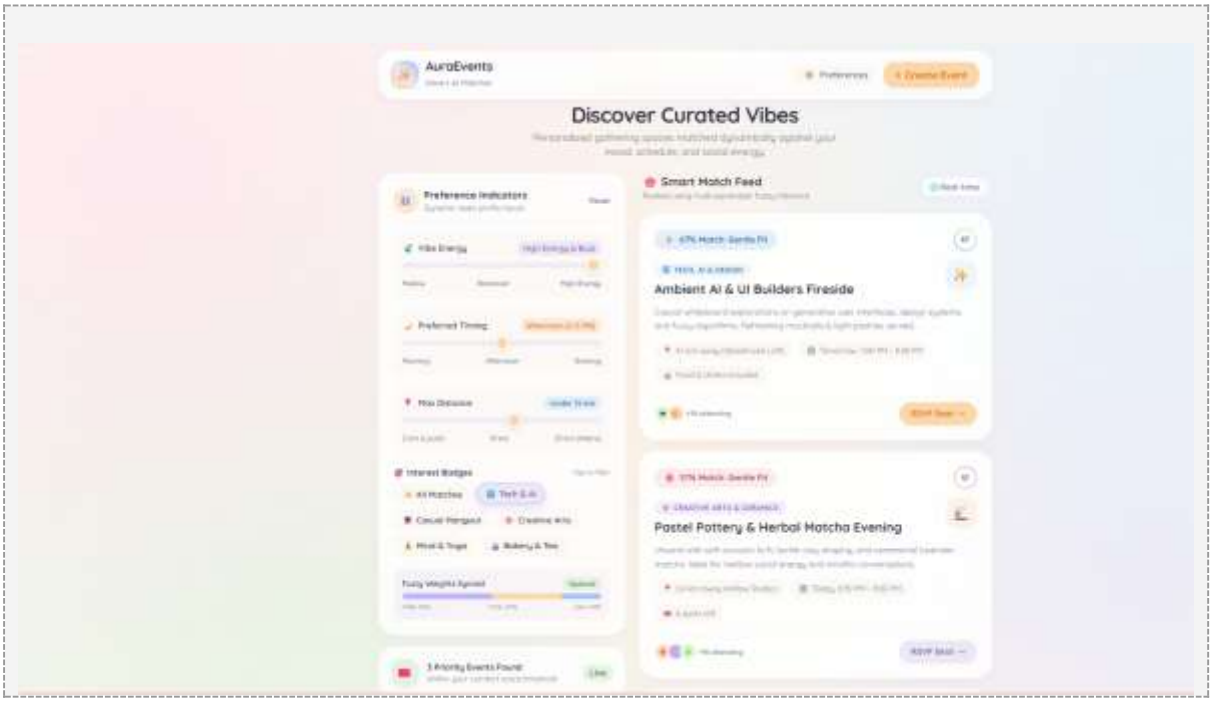


Fig. 3.2 Dashboard after changing preferences (cards re-ranked by the fuzzy engine)

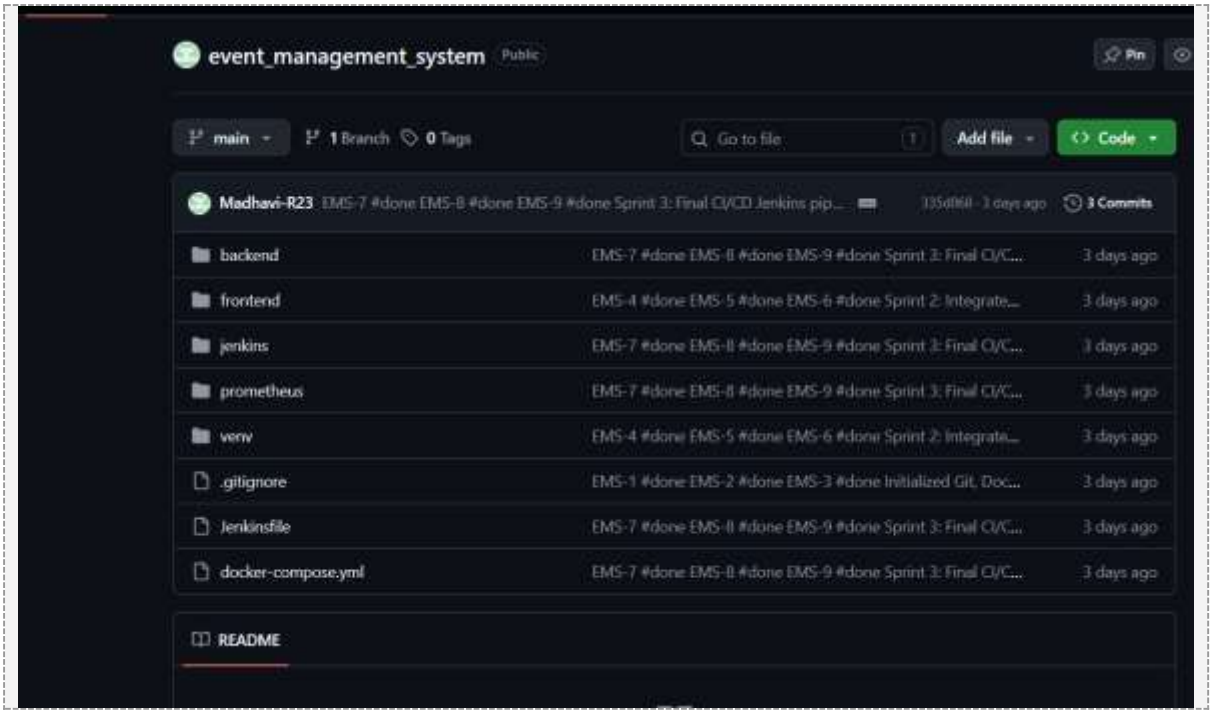


Fig. 3.3 JIRA adapted with github



Fig. 3.4 Prometheus target status and the total_matches_requested_total query

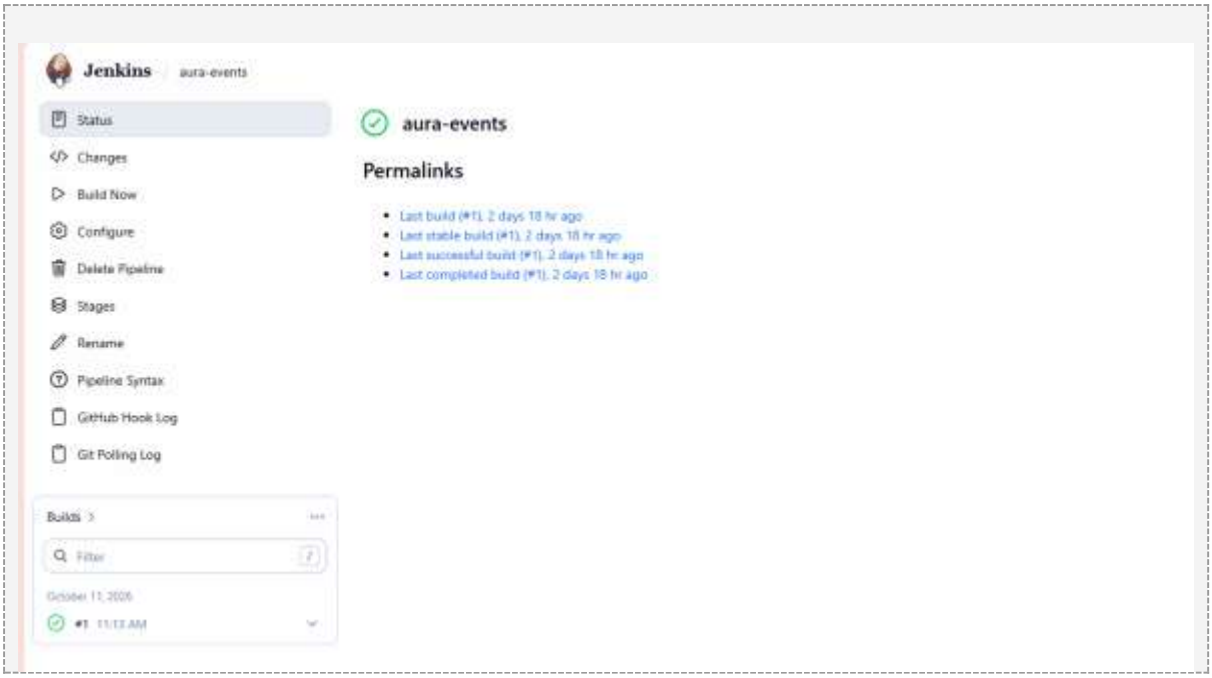


Fig. 3.5 Jenkins pipeline build triggered by a GitHub push

3.2 Discussion

Fuzzy versus crisp matching. In scenarios S1 to S3 the crisp filter found exactly one event, while the fuzzy engine returned the same event on top and also ranked the other two, so the attendee still sees alternatives. In S4 and S5 no event satisfied every crisp condition and the crisp filter returned nothing, whereas the fuzzy engine produced a full ranking in which the best match (66% and 65%) was clearly separated from the poor ones (11% and 47%).

Graceful behavior. Table 3.2 shows that the scores change smoothly with the distance limit. The pottery evening, 2.4 km away, still scores 88% when the limit is 2 km, where a crisp filter would drop it completely, and its score rises gradually as the limit is relaxed. This is the behavior that motivated the use of fuzzy sets. The interest badge works as a soft preference: events outside the chosen category are scaled down to about 80% of their score rather than hidden.

DevOps outcomes. Every push to the main branch can rebuild and redeploy the stack without manual steps, and the verification stage fails the build if the application does not become healthy. Prometheus makes the system observable: the request counter shows how often matches are requested for each interest, and the histogram records how long the fuzzy step takes. Several practical issues had to be solved along the way, namely the missing Docker client in the stock Jenkins image, host-path bind mounts that do not exist inside the Jenkins workspace, and the risk of the pipeline recreating its own Jenkins container.

Agile outcomes. Each sprint ended with a working increment: a running stack and dashboard (Sprint 1), a working matching API (Sprint 2) and automated delivery with monitoring (Sprint 3). Sprint 3 also showed the benefit of iterating, because the Docker and Compose files from Sprint 1 were revised once the real deployment requirements became clear.

Limitations. The event database is a hardcoded set of three mock events and their distances are fixed rather than computed from the user's location. The weights and membership constants were chosen by the team and have not been validated with real users. The "Events Processed" and "Match Accuracy" figures in the dashboard footer are static placeholders, not measured values; only the inference time shown there is measured. Finally, because Jenkins runs on a local machine, GitHub can reach it only through a public tunnel, which is why a polling trigger is kept as a fallback.

Chapter 4
Conclusion

4.1 Conclusion

This project designed and developed AuraEvents, a smart event matcher that applies fuzzy logic to a problem where crisp filters fall short. Imprecise preferences for vibe, timing, distance and interest are converted into membership degrees and aggregated into a percentage match, so that attendees always receive a ranked list with a clear confidence for each event, even when nothing fits perfectly. In the test scenarios the fuzzy approach returned a complete ranking in every case, while the crisp baseline returned a single event or none.

The system was built iteratively with Agile Scrum in three sprints, packaged with Docker Compose, delivered through a Jenkins pipeline triggered by GitHub pushes and observed through Prometheus metrics. The project shows that soft computing techniques and Agile and DevOps practices can be combined effectively to build practical, easy-to-understand and continuously deliverable software.

4.2 Future Scope

- Replace the weighted-average engine with a full Mamdani rule base (for example using scikit-fuzzy) and tune the membership functions from real usage data using neuro-fuzzy (ANFIS) or type-2 fuzzy sets.
- Replace the mock events with a database (for example PostgreSQL), add event creation by organizers and persistent RSVPs.
- Compute distances from the attendee's real location and integrate calendar data to check schedule conflicts.
- Extend the system with organizer-side modules for volunteer management, task priority and event risk using fuzzy rules.
- Add Grafana dashboards and alerting rules on the Prometheus metrics, and a test stage in the Jenkins pipeline.
- Deploy the pipeline to a cloud platform with an image registry and role-based access control.

Chapter 5
References

[1] L. A. Zadeh, “Fuzzy sets,” *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.

[2] E. H. Mamdani and S. Assilian, “An experiment in linguistic synthesis with a fuzzy logic controller,” *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.

[3] K. Schwaber and J. Sutherland, *The Scrum Guide*, 2020. [Online]. Available: https://scrumguides.org

[4] K. Beck et al., “Manifesto for Agile Software Development,” 2001. [Online]. Available: https://agilemanifesto.org

[5] T. J. Ross, *Fuzzy Logic with Engineering Applications*, 4th ed. Wiley, 2016.

[6] H. Takeuchi and I. Nonaka, “The New New Product Development Game,” *Harvard Business Review*, vol. 64, no. 1, pp. 137–146, 1986.

[7] T. Takagi and M. Sugeno, “Fuzzy identification of systems and its applications to modeling and control,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-15, no. 1, pp. 116–132, 1985.

[8] FastAPI documentation. [Online]. Available: https://fastapi.tiangolo.com (accessed Oct. 2026).

[9] Prometheus documentation. [Online]. Available: https://prometheus.io/docs/ (accessed Oct. 2026).

[10] Jenkins User Documentation: Pipeline. [Online]. Available: https://www.jenkins.io/doc/book/pipeline/ (accessed Oct. 2026).

[11] Docker Compose documentation. [Online]. Available: https://docs.docker.com/compose/ (accessed Oct. 2026).